Meu primeiro CRUD com HTML, PHP e MySQL



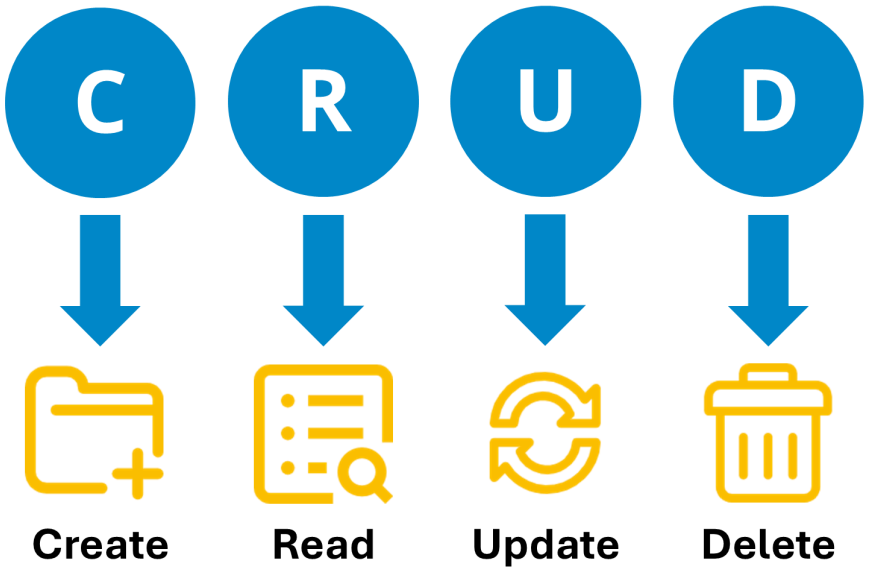
Criado em outubro de 2026 por
Lorenzo Jordani Bertozzi Luz

Introdução **2**
O que é uma página web e como ela é construída **3**
Front-end, back-end e banco de dados **3**
Preparando o ambiente de desenvolvimento **4**
1.Iniciar o XAMPP 5
2.Criar a pasta do projeto 5
3.Testando o PHP 6
Criando o banco de dados MySQL **6**
Inserindo registros de teste e consultando com SELECT 7
Inserir dados (INSERT) 7
Consultar dados (SELECT) 8
Atualizar e excluir (UPDATE e DELETE) 9
Limpar os dados de teste 9
Organização dos arquivos **9**
Conectando o PHP ao MySQL **10**
Arquivos de base: funcoes.php, topo.php, rodape.php e style.css **12**
funcoes.php 12
topo.php e rodape.php 14
topo.php 14
rodape.php 14
style.css 15
Cadastro, login, sessão e logout 19
cadastro.php 19
login.php 22
logout.php 24
O CRUD de tarefas 25
index.php (Read) 25
form_tarefa.php (formulário compartilhado) 27
criar.php (Create) 28
editar.php (Update) 29
excluir.php (Delete) 30
Testando tudo 31

Introdução

Neste guia você construirá, passo a passo, um sistema de tarefas com cadastro, login e **CRUD** completo, usando **HTML, PHP e MySQL** no seu próprio computador.

CRUD é uma sigla formada pelas quatro operações fundamentais de manipulação de dados: **C - Create (Criar)** onde iremos cadastrar um novo aluno no sistema; **R - Read (Ler)** onde podemos consultar e listar os alunos cadastrados; **U - Update (Atualizar)** permitindo assim modificar o nome, e-mail ou curso de um aluno; e por fim **D - Delete (Excluir)** permitindo remover um aluno do banco de dados.



Essas quatro operações aparecem em praticamente todo tipo de sistema: cadastro de usuários, gerenciamento de produtos, controle de estoque, bibliotecas, sistemas acadêmicos e muito mais.

Nosso objetivo será desenvolver um projeto pequeno, mas com uma estrutura que permita evoluir para aplicações maiores.

O projeto final terá estas funcionalidades:

- Cadastro de usuário, com a senha guardada como hash (nunca em texto puro)
- Login, sessão que mantém o usuário conectado e logout
- Lista de tarefas visível só para o dono delas
- Criar, editar e excluir tarefas
- Visual organizado em um único arquivo `style.css`

O que você precisa: o VS Code para escrever o código e o XAMPP (Apache e MySQL, com o phpMyAdmin) para rodar tudo localmente. Não é preciso instalar mais nada.

Como usar o guia: siga as seções na ordem. Toda seção termina em algo que você consegue ver funcionando no navegador, e os códigos estão completos, prontos para copiar.

Materiais para consulta: Para estudar os conceitos utilizados neste guia, consulte a documentação oficial:

- PHP — Manual oficial do PHP.
- MySQL — Documentação do MySQL.
- Apache HTTP Server — Documentação oficial.
- XAMPP — Site oficial do XAMPP.
- Visual Studio Code — Documentação do VS Code.
- W3Schools — PHP Tutorial

O que é uma página web e como ela é construída

Uma página web é um arquivo de texto (HTML) que um servidor entrega ao navegador quando ele pede. Tudo o que você vê na tela nasce desse pedido e desta resposta.

O ciclo é sempre o mesmo:

1. Você digita um endereço (URL) ou clica em um link, e o navegador envia uma requisição HTTP ao servidor.
2. O servidor recebe o pedido e decide o que responder.
3. Ele devolve uma resposta, quase sempre um documento HTML.
4. O navegador lê o HTML, aplica o CSS, executa o JavaScript e desenha a página.

Três linguagens montam a parte que o navegador entende:

- **HTML** define a estrutura e o conteúdo: títulos, parágrafos, formulários, tabelas.
- **CSS** define a aparência: cores, tamanhos, espaçamento, layout.
- **JavaScript** adiciona comportamento no navegador. Neste guia quase não precisaremos dele.

Uma página de HTML puro é **estática**: todo mundo vê o mesmo arquivo. Para mostrar dados que mudam (as tarefas de cada usuário), o servidor precisa **gerar** o HTML na hora. É aí que entra o PHP: ele roda no servidor, consulta o banco e monta o HTML antes de enviar.

O navegador nunca recebe o código PHP, só o HTML pronto. Por isso, senhas e regras de negócio ficam protegidas no servidor.

Nas requisições, usaremos dois métodos HTTP:

Método	Uso típico
GET	Pedir uma página ou dados; os parâmetros aparecem na URL
POST	Enviar dados de um formulário; eles vão no corpo da requisição

Front-end, back-end e banco de dados

Um sistema web divide o trabalho em três camadas, e o navegador só conversa com o servidor, nunca diretamente com o banco de dados.

O navegador nunca fala direto com o banco de dados



O pedido vai do navegador ao PHP; o PHP consulta o MySQL e devolve ao navegador uma página HTML pronta.

Camada	Onde roda	Onde roda	Responsabilidade
Front-end	No navegador	HTML e CSS	Mostrar as telas e receber os dados digitados nos formulários
Back-end	No servidor	PHP	Validar dados, aplicar regras, controlar login e sessão, conversar com o banco
Banco de dados	No servidor	MySQL e SQL	Guardar os dados de forma permanente e responder consultas

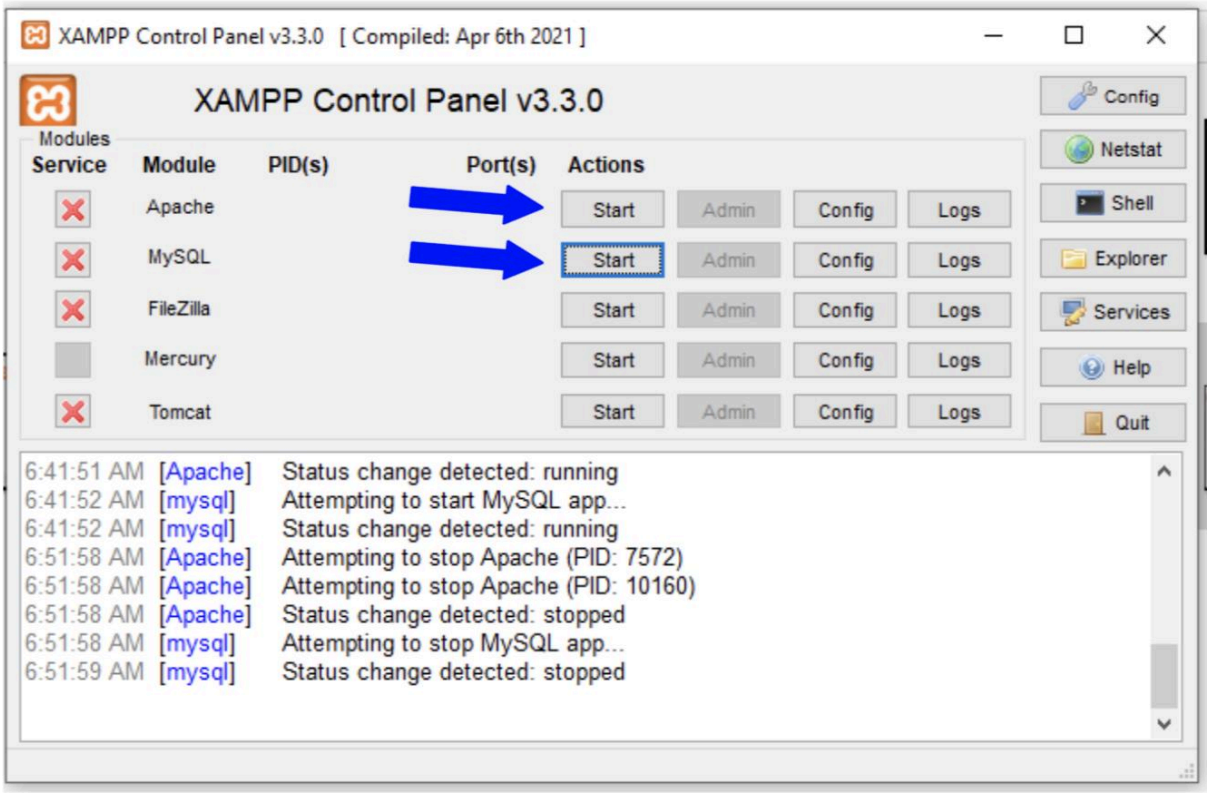
Exemplo no nosso projeto: ao enviar o formulário de login, o navegador manda e-mail e senha para o POST. O PHP busca o usuário no MySQL com um SELECT, confere o hash da senha e responde com a página certa.

Preparando o ambiente de desenvolvimento

O ambiente tem três peças: o XAMPP (servidor Apache e banco MySQL), o navegador e o VS Code. Tudo começa iniciando o XAMPP.

1.Iniciar o XAMPP

1. **Abra o XAMPP** Control Panel (no Windows, pelo menu Iniciar).
2. **Clique em Start** nas linhas **Apache** e **MySQL**. Os nomes ficam com fundo verde quando estão rodando.
3. Se o Apache não subir, outro programa provavelmente usa a porta 80 (Skype, IIS, outro servidor). Feche-o ou troque a porta em Config > Apache (httpd.conf).



Abra o navegador e acesse:

- `http://localhost/` — verifica se o servidor web está acessível.
- `http://localhost/phpmyadmin/` — abre a ferramenta de administração do banco de dados.

Se algum serviço não iniciar, pode existir um conflito de portas ou outro serviço em execução. Resolva isso antes de continuar.

2.Criar a pasta do projeto

O Apache só funciona para arquivos que estejam dentro da pasta `htdocs` do XAMPP. No Windows ela fica em `C:\xampp\htdocs`; no Linux, `/opt/lampp/htdocs`; no macOS, `/Applications/XAMPP/htdocs`.

1. Dentro de `htdocs`, crie a pasta `crud_tarefas` (minúsculas, sem espaços nem acentos).

2. No VS Code, vá em **File > Open Folder** e selecione crud_tarefas. Todos os arquivos do guia ficam nela.
3. Opcional: instale a extensão **PHP Intelephense** no VS Code para ter autocompletar e destaque de erros em PHP.

O endereço do seu projeto será http://localhost/crud_tarefas/.

3.Testando o PHP

Um arquivo de poucas linhas confirma que o Apache está executando PHP.

Passo 1 - No VS Code, crie o arquivo teste.php dentro de crud_tarefas com este conteúdo:

```php
<?php
    echo "<h1>PHP funcionando!</h1>";
    echo "<p>Versão do PHP: " . phpversion() . "</p>";
    echo "<p>Data e hora do servidor: " . date('d/m/Y H:i:s') . "</p>";
?>
```

Passo 2 - Abra http://localhost/crud_tarefas/teste.php no navegador.
Passo 3 - Você deve ver o título, a versão do PHP e a data e hora atuais. Se aparecer, o PHP está rodando.

Todo código PHP fica entre <?php e ?>. Em arquivos que só têm PHP, como este, o ?> final é omitido de propósito.

Criando o banco de dados MySQL

O banco guarda duas tabelas relacionadas: usuarios (quem acessa o sistema) e tarefas (cada tarefa pertence a um usuário).

1. Com o MySQL iniciado no XAMPP, abra http://localhost/phpmyadmin.
2. Clique na aba **SQL**, no topo da tela.
3. Cole o script abaixo e clique em **Executar**.

```sql
CREATE DATABASE IF NOT EXISTS crud_tarefas
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE crud_tarefas;

CREATE TABLE IF NOT EXISTS usuarios (
  id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  email     VARCHAR(150) NOT NULL UNIQUE,
  senha VARCHAR(255) NOT NULL
);


CREATE TABLE IF NOT EXISTS tarefas (
  id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  usuario_id    INT UNSIGNED NOT NULL,
```

```
  titulo        VARCHAR(150) NOT NULL,
  descricao     TEXT NULL,
  status        ENUM('pendente', 'andamento', 'concluida') NOT NULL
DEFAULT 'pendente',
  criado_em     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  CONSTRAINT fk_tarefas_usuario
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
    ON DELETE CASCADE
);
```

4. Na lateral esquerda do phpMyAdmin, o banco `crud_tarefas` deve aparecer com as tabelas `usuarios` e `tarefas`.

O que cada parte do script significa:

Elemento	Significado
`utf8mb4`	Conjunto de caracteres que aceita acentos e emojis
`AUTO_INCREMENT PRIMARY KEY`	Identificador único, gerado automaticamente a cada registro novo
`UNIQUE` no e-mail	Impede dois usuários com o mesmo e-mail
`senha VARCHAR(255)`	Guarda o hash da senha, nunca a senha digitada; 255 caracteres dão folga para algoritmos futuros
`ENUM(...)` no status	Limita o status a três valores permitidos
`FOREIGN KEY ... ON DELETE CASCADE`	Liga cada tarefa a um usuário; ao apagar o usuário, as tarefas dele são apagadas junto
`ON UPDATE CURRENT_TIMESTAMP`	Atualiza `atualizado_em` sozinho sempre que a linha muda

Guarde o script também em um arquivo `banco.sql` na pasta do projeto: ele serve de backup e permite recriar o banco em outro computador.

Inserindo registros de teste e consultando com SELECT

Antes de ligar o PHP ao banco, vale praticar o SQL direto no phpMyAdmin, para entender o que o PHP fará por você mais adiante.

Inserir dados (INSERT)

Na aba **SQL**, execute:

```
USE crud_tarefas;
```

```sql
INSERT INTO usuarios (nome, email, senha)
VALUES ('Usuário de Teste', 'teste@exemplo.com','senha');


INSERT INTO tarefas (usuario_id, titulo, descricao, status) VALUES
(1, 'Instalar o XAMPP', 'Baixar e iniciar Apache e MySQL',
'concluida'),
(1, 'Criar o banco de dados', 'Rodar o script SQL no phpMyAdmin',
'andamento'),
(1, 'Fazer o primeiro CRUD', 'Listar, criar, editar e excluir tarefas',
'pendente');
```

O valor `1` em `usuario_id` é o `id` do usuário recém-criado. O `id` das tarefas é gerado sozinho.

Consultar dados (SELECT)

Ainda na aba **SQL**, execute:

```sql
-- Todas as colunas e todas as linhas
SELECT * FROM usuarios;
SELECT * FROM tarefas;


-- Só algumas colunas, com filtro
SELECT id, titulo, status FROM tarefas WHERE status = 'pendente';


-- Ordenando: mais recentes primeiro
SELECT * FROM tarefas WHERE usuario_id = 1 ORDER BY criado_em DESC;


-- Juntando as duas tabelas
SELECT t.titulo, t.status, u.nome
FROM tarefas t
INNER JOIN usuarios u ON u.id = t.usuario_id;
```

Leia `SELECT` colunas `FROM` tabela `WHERE` condição `ORDER BY` coluna como: "traga estas colunas, desta tabela, só as linhas que atendem à condição, nesta ordem". O `*` significa todas as colunas.

Atualizar e excluir (UPDATE e DELETE)

```sql
UPDATE tarefas SET status = 'concluida' WHERE id = 2;

DELETE FROM tarefas WHERE id = 3;
```

Sempre use `WHERE` em `UPDATE` e `DELETE`. Sem ele, o comando atinge todas as linhas da tabela.

Limpar os dados de teste

O usuário de teste tem um hash falso e não consegue fazer login. Apague-o antes de seguir, para que os próximos registros comecem do `id` 1:

```sql
DELETE FROM tarefas;

DELETE FROM usuarios;

ALTER TABLE tarefas AUTO_INCREMENT = 1;

ALTER TABLE usuarios AUTO_INCREMENT = 1;
```

Organização dos arquivos

O projeto tem 13 arquivos na pasta `crud_tarefas`, cada um com uma única função, para que você saiba onde mexer quando algo precisa mudar.

```
crud_tarefas/
├── conexao.php    conexão com o MySQL
├── funcoes.php    sessão e funções auxiliares
├── topo.php       início do HTML (cabeçalho e menu)
├── rodape.php     fim do HTML
├── style.css      estilo de todas as páginas
├── cadastro.php   criar conta
├── login.php      entrar
├── logout.php     sair
├── index.php      listar tarefas (Read)
├── criar.php      nova tarefa (Create)
├── editar.php     alterar tarefa (Update)
├── excluir.php    remover tarefa (Delete)
└── form_tarefa.php   campos do formulário de tarefa
```

Arquivo	O que faz	Precisa de login?
`conexao.php`	Abre a conexão com o MySQL; é incluído pelas páginas que usam o banco	Não é uma página
`funcoes.php`	Inicia a sessão e reúne funções usadas em todo lugar	Não é uma página
`topo.php` e `rodape.php`	Abrem e fecham o HTML comum a todas as telas	Não são páginas

`form_tarefa.php`	Formulário compartilhado por criar.php e editar.php	Não é uma página
`style.css`	Cores, espaçamentos e layout	Não
`cadastro.php`	Formulário de cadastro; grava o usuário com a senha em hash	Não
`login.php`	Confere e-mail e senha e abre a sessão	Não
`logout.php`	Encerra a sessão	Sim
`index.php`	Lista as tarefas do usuário	Sim
`criar.php`, `editar.php`, `excluir.php`	Inserem, alteram e removem tarefas	Sim

O caminho que o usuário percorre:

1. Visitante abre `index.php`, não tem sessão e é enviado para `login.php`.
2. Sem conta, vai a `cadastro.php`, cria a conta e volta ao login.
3. Após o login, a sessão é aberta e ele cai em `index.php`.
4. Dali segue para `criar.php`, `editar.php` ou `excluir.php` e sempre volta à lista.
5. Em `logout.php` a sessão é encerrada e ele volta ao login.

Uma regra vale para todas as páginas: primeiro a lógica (sessão, banco, redirecionamentos), depois o HTML. Um `header('Location: ...')` só funciona se nada tiver sido enviado ao navegador antes dele.

Conectando o PHP ao MySQL

O arquivo `conexao.php` cria um objeto de conexão chamado `$conn`, que as outras páginas reutilizam com `require_once`. Crie-o na pasta do projeto:

```php
<?php

$host  = 'localhost';
$user  = 'root';
$pass  = '';            // padrão do XAMPP: sem senha
$banco = 'crud_tarefas';

// Faz o mysqli lançar exceções em vez de avisos silenciosos
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

try {
```

```
    $conn = new mysqli($host, $user, $pass, $banco);
    $conn->set_charset('utf8mb4');
} catch (mysqli_sql_exception $e) {
    // Em um sistema real, grave o erro em log e não mostra detalhes ao usuário
    die('Erro ao conectar ao banco de dados: ' . $e->getMessage());
}
```

Para testar, troque o conteúdo de `teste.php` por este código e abra `http://localhost/crud_tarefas/teste.php`. Se a conexão funcionar, ele lista as tarefas que você criou antes

(se tiver limpado os dados de teste, a lista aparece vazia, o que também é sucesso).

```
<?php
require_once 'conexao.php';

$resultado = $conn->query('SELECT id, titulo, status FROM tarefas');

echo '<h1>Tarefas no banco</h1><ul>';
while ($linha = $resultado->fetch_assoc()) {
        echo '<li>'. htmlspecialchars($linha['titulo']) . ' (' . $linha['status'] . ')</li>';
}
echo '</ul>';
```

O que cada trecho faz:

Trecho	Função
`new mysqli(...)`	Abre a conexão com servidor, usuário, senha e banco
`set_charset('utf8mb4')`	Garante que acentos cheguem e saiam corretos
`require_once`	Inclui o arquivo uma única vez, evitando conexões duplicadas
`query()`	Executar um SQL e devolve o resultado
`fetch_assoc()`	Devolve a próxima linha como array ($linha['titulo']) ou null ao acabar
`htmlspecialchars()`	Converte caracteres como < e > em texto seguro para o HTML

Consultas seguras: prepared statements: Nunca monte um SQL concatenando dados do usuário, como em "... WHERE email = '`. $email . "'". Quem digita um texto malicioso consegue alterar o comando (ataque chamado SQL Injection). A forma correta separa o comando dos dados:

```
$stmt = $conn->prepare('SELECT id, titulo FROM tarefas WHERE usuario_id
= ? AND status = ?');

$stmt->bind_param('is', $usuarioId, $status);   // i = inteiro, s =
string

$stmt->execute();

$resultado = $stmt->get_result();
```

O `?` marca onde entra um valor, e **bind_param** informa o tipo de cada um na mesma ordem:

Letra	Tipo
`i`	Inteiro (`id`, `usuario_id`)
`s`	String(nome, e-mail, título)
`d`	Decimal

Todas as consultas do projeto usam esse padrão.

Arquivos de base: funcoes.php, topo.php, rodape.php e style.css

Antes das telas, crie os quatro arquivos que todas elas compartilham. Assim o cabeçalho, o rodapé e as regras comuns ficam escritos uma vez só.

funcoes.php

Inicia a sessão e guarda as funções de uso geral: checar login, exigir login, escapar texto para o HTML e validar uma tarefa.

```
<?php

// Inicia a sessão uma única vez (necessário para o $_SESSION
funcionar)

if (session_status() === PHP_SESSION_NONE) {

    session_start();

}

// Status permitidos para uma tarefa: valor no banco => texto exibido

const STATUS_TAREFA = [
```

```
    'pendente'  => 'Pendente',
    'andamento' => 'Em andamento',
    'concluida' => 'Concluída',
];
// Retorna true se existe um usuário logado
function esta_logado(): bool
{
    return isset($_SESSION['usuario_id']);
}
// Envia para o login quem não estiver logado
function exigir_login(): void
{
    if (!esta_logado()) {
        header('Location: login.php');
        exit;
    }
}
// Escapa um texto antes de exibi-lo no HTML (protege contra XSS)
function e(?string $texto): string
{
    return htmlspecialchars($texto ?? '', ENT_QUOTES, 'UTF-8');
}
// Valida os dados de uma tarefa e devolve a lista de erros (vazia se
estiver tudo certo)
function validar_tarefa(array $dados): array
{
    $erros = [];

    if (empty($dados['titulo'])) {
        $erros[] = 'Informe o título da tarefa.';
    } elseif (mb_strlen($dados['titulo']) > 150) {
        $erros[] = 'O título pode ter no máximo 150 caracteres.';
    }

    if (!array_key_exists($dados['status'], STATUS_TAREFA)) {
```

```
        $erros[] = 'Status inválido.';
    }
    return $erros;
}
```

topo.php e rodape.php

O topo.php abre o HTML, carrega o style.css e mostra o menu, que muda conforme o usuário estiver logado ou não. O rodape.php fecha o que o topo abriu.

topo.php

Início do HTML. Define o título Página antes de incluir este arquivo.

```
<?php
require_once __DIR__ . '/funcoes.php';
$tituloPagina = $tituloPagina ?? 'Tarefas';?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title><?= e($tituloPagina) ?> | Minhas Tarefas</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<header class="topo">
    <div class="container topo-conteudo">
        <a class="marca" href="index.php">Minhas Tarefas</a>
        <nav>
            <?php if (esta_logado()): ?>
                <span class="saudacao">Olá, <?=
e($_SESSION['usuario_nome']) ?></span>
                <a href="index.php">Tarefas</a>
                <a href="logout.php">Sair</a>
            <?php else: ?>
                <a href="login.php">Entrar</a>
                <a href="cadastro.php">Cadastrar</a>
            <?php endif; ?>
        </nav>
    </div>
</header>
<main class="container">
```

rodape.php

Final do HTML. Define o rodapé da página antes de incluir este arquivo.

```
</main>
<footer class="rodape">
    <div class="container">Meu primeiro CRUD com PHP e MySQL</div>
</footer>
</body>
</html>
```

A sintaxe <?= ... ?> é um atalho para <?php echo ... ?>. A forma if (...): ... endif; deixa o PHP legível quando misturado com HTML.

style.css

Este bloco do código definirá um estilo compartilhado por todas as páginas

```
:root {
    --cor-primaria: #2563eb;
    --cor-primaria-escura: #1d4ed8;
    --cor-perigo: #dc2626;
    --cor-perigo-escura: #b91c1c;
    --cor-fundo: #f3f4f6;
    --cor-card: #ffffff;
    --cor-texto: #1f2937;
    --cor-suave: #6b7280;
    --cor-borda: #e5e7eb;
    --raio: 8px;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    font-family: system-ui, -apple-system, 'Segoe UI', Roboto,
sans-serif;
    background: var(--cor-fundo);
    color: var(--cor-texto);
    line-height: 1.5;
}
```

```
main { flex: 1; padding: 32px 0; }


.container { width: 100%; max-width: 960px; margin: 0 auto; padding: 0
16px; }


a { color: var(--cor-primaria); text-decoration: none; }

a:hover { text-decoration: underline; }


h1 { margin: 0 0 16px; font-size: 1.6rem; }


/* Topo */

.topo { background: var(--cor-card); border-bottom: 1px solid
var(--cor-borda); }

.topo-conteudo { display: flex; align-items: center; justify-content:
space-between; min-height: 60px; }

.marca { font-weight: 700; font-size: 1.15rem; color: var(--cor-texto);
}

.marca:hover { text-decoration: none; }

.topo nav { display: flex; align-items: center; gap: 16px; }

.saudacao { color: var(--cor-suave); }


/* Cards */

.card {

    background: var(--cor-card);

    border: 1px solid var(--cor-borda);

    border-radius: var(--raio);

    padding: 24px;

    box-shadow: 0 1px 2px rgba(0, 0, 0, .04);

}

.card-auth { max-width: 420px; margin: 0 auto; }

.vazio { text-align: center; color: var(--cor-suave); }


/* Cabeçalho de página */

.cabecalho-pagina { display: flex; align-items: center;
justify-content: space-between; gap: 16px; margin-bottom: 16px; }

.cabecalho-pagina h1 { margin: 0; }
```

```
/* Formulários */
.campo { margin-bottom: 16px; }
.campo label { display: block; margin-bottom: 6px; font-weight: 600;
font-size: .9rem; }
.campo input, .campo textarea, .campo select {
    width: 100%;
    padding: 10px 12px;
    font: inherit;
    color: inherit;
    background: #fff;
    border: 1px solid var(--cor-borda);
    border-radius: var(--raio);
}
.campo input:focus, .campo textarea:focus, .campo select:focus {
    outline: 2px solid var(--cor-primaria);
    outline-offset: 1px;
    border-color: var(--cor-primaria);
}
.form-acoes { display: flex; gap: 8px; }
.texto-form { margin: 16px 0 0; text-align: center; color:
var(--cor-suave); font-size: .9rem; }


/* Botões */
.btn {
    display: inline-block;
    padding: 10px 16px;
    font: inherit;
    font-weight: 600;
    text-align: center;
    border: 1px solid transparent;
    border-radius: var(--raio);
    cursor: pointer;
}
.btn:hover { text-decoration: none; }
.btn-primario { background: var(--cor-primaria); color: #fff; }
```

```
.btn-primario:hover { background: var(--cor-primaria-escura); }

.btn-secundario { background: #fff; color: var(--cor-texto);
border-color: var(--cor-borda); }

.btn-secundario:hover { background: var(--cor-fundo); }

.btn-perigo { background: var(--cor-perigo); color: #fff; }

.btn-perigo:hover { background: var(--cor-perigo-escura); }

.btn-pequeno { padding: 6px 10px; font-size: .85rem; }

.btn-bloco { width: 100%; }


/* Alertas */

.alerta { padding: 12px 16px; margin-bottom: 16px; border: 1px solid;
border-radius: var(--raio); }

.alerta ul { margin: 0; padding-left: 20px; }

.alerta-erro { background: #fef2f2; border-color: #fecaca; color:
#991b1b; }

.alerta-sucesso { background: #f0fdf4; border-color: #bbf7d0; color:
#166534; }


/* Tabela de tarefas */

.tabela { width: 100%; border-collapse: collapse; }

.tabela th, .tabela td { padding: 12px 8px; text-align: left;
vertical-align: top; border-bottom: 1px solid var(--cor-borda); }

.tabela th { font-size: .8rem; text-transform: uppercase;
letter-spacing: .04em; color: var(--cor-suave); }

.tabela tbody tr:last-child td { border-bottom: none; }

.descricao { margin-top: 4px; font-size: .9rem; color:
var(--cor-suave); }

.acoes { white-space: nowrap; }

.acoes form { display: inline; }


/* Etiquetas de status */

.badge { display: inline-block; padding: 2px 10px; font-size: .8rem;
font-weight: 600; border-radius: 999px; }

.badge-pendente { background: #fef3c7; color: #92400e; }

.badge-andamento { background: #dbeafe; color: #1e40af; }

.badge-concluida { background: #dcfce7; color: #166534; }


/* Rodapé */
```

```
.rodape { padding: 16px 0; text-align: center; font-size: .85rem;
color: var(--cor-suave); }


/* Telas pequenas */

@media (max-width: 640px) {

    .topo-conteudo { flex-direction: column; gap: 8px; padding: 12px
16px; }

    .cabecalho-pagina { flex-direction: column; align-items: stretch; }

    .tabela thead { display: none; }

    .tabela, .tabela tbody, .tabela tr, .tabela td { display: block;
width: 100%; }

    .tabela tr { padding: 12px 0; border-bottom: 1px solid
var(--cor-borda); }

    .tabela td { padding: 4px 0; border: none; }

}
```

O CSS começa com variáveis em :root (cores, raio das bordas). Para mudar a identidade visual do sistema, basta alterar esses valores. As classes .btn, .card, .alerta e .badge são blocos reutilizáveis que as telas combinam.

Cadastro, login, sessão e logout

O cadastro grava o usuário com a senha transformada em hash, o login confere essa senha e abre uma sessão, e o logout a encerra. As explicações vêm logo após cada código.

Conceitos antes do código

Conceito	O que é	Função PHP
Hash de senha	Transformação de mão única: do hash não se recupera a senha original. O banco guarda só o hash	password_hash() e password_verify()
Sessão	Memória do servidor sobre um visitante, ligada ao navegador por um cookie. É como o sistema "lembra" quem está logado	session_start() e $_SESSION
ID da sessão	Troca o identificador da sessão após o login, dificultando o roubo de sessões	session_regenerate_id(true)

cadastro.php

```
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
```

```php
// Quem já está logado não precisa se cadastrar
if (esta_logado()) {
    header('Location: index.php');
    exit;
}

$erros = [];
$nome  = '';
$email = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nome   = trim($_POST['nome'] ?? '');
    $email  = trim($_POST['email'] ?? '');
    $senha  = $_POST['senha'] ?? '';
    $senha2 = $_POST['senha2'] ?? '';

    // 1) Validação
    if ($nome === '') {
        $erros[] = 'Informe seu nome.';
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $erros[] = 'Informe um e-mail válido.';
    }
    if (strlen($senha) < 6) {
        $erros[] = 'A senha deve ter pelo menos 6 caracteres.';
    }
    if ($senha !== $senha2) {
        $erros[] = 'As senhas não conferem.';
    }

    // 2) O e-mail já está em uso?
    if (empty($erros)) {
        $stmt = $conn->prepare('SELECT id FROM usuarios WHERE email =
?');
        $stmt->bind_param('s', $email);
        $stmt->execute();
        $stmt->store_result();
        if ($stmt->num_rows > 0) {
            $erros[] = 'Este e-mail já está cadastrado.';
        }
        $stmt->close();
    }

    // 3) Grava o usuário com a senha em hash
    if (empty($erros)) {
        $hash = password_hash($senha, PASSWORD_DEFAULT);
        $stmt = $conn->prepare('INSERT INTO usuarios (nome, email,
senha) VALUES (?, ?, ?)');
        $stmt->bind_param('sss', $nome, $email, $hash);
        $stmt->execute();
        $stmt->close();

        header('Location: login.php?cadastro=ok');
        exit;
    }
}

$tituloPagina = 'Cadastro';
```

```
require 'topo.php';
?>
<section class="card card-auth">
    <h1>Criar conta</h1>

    <?php if (!empty($erros)): ?>
        <div class="alerta alerta-erro">
            <ul>
                <?php foreach ($erros as $erro): ?>
                    <li><?= e($erro) ?></li>
                <?php endforeach; ?>
            </ul>
        </div>
    <?php endif; ?>

    <form method="post" action="cadastro.php">
        <div class="campo">
            <label for="nome">Nome</label>
            <input type="text" id="nome" name="nome" value="<?=
e($nome) ?>" required>
        </div>
        <div class="campo">
            <label for="email">E-mail</label>
            <input type="email" id="email" name="email" value="<?=
e($email) ?>" required>
        </div>
        <div class="campo">
            <label for="senha">Senha</label>
            <input type="password" id="senha" name="senha"
minlength="6" required>
        </div>
        <div class="campo">
            <label for="senha2">Repita a senha</label>
            <input type="password" id="senha2" name="senha2"
minlength="6" required>
        </div>
        <button type="submit" class="btn btn-primario
btn-bloco">Cadastrar</button>
    </form>

    <p class="texto-form">Já tem conta? <a
href="login.php">Entrar</a></p>
</section>
<?php require 'rodape.php'; ?>
```

Como funciona:

1. A mesma página serve de formulário (pedido GET) e de receptora dos dados (pedido POST); `$_SERVER['REQUEST_METHOD']` diz qual dos dois está acontecendo.
2. `trim()` remove espaços nas pontas, e `?? ''` evita erros se o campo não vier.
3. Os erros são juntados em `$erros`. Se a lista ficar vazia, o cadastro segue.
4. `password_hash($senha, PASSWORD_DEFAULT)` gera o hash (algo como `$2y$10$...`), diferente a cada chamada, mesmo para a mesma senha.
5. O formulário reexibe `nome` e `email` quando há erro, para o usuário não digitar tudo de novo. A senha nunca é reexibida.

6. Após gravar, header('Location: ...') redireciona; o exit logo depois impede que o resto do script continue.

login.php

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';

if (esta_logado()) {
    header('Location: index.php');
    exit;
}

$erro  = '';
$email = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = trim($_POST['email'] ?? '');
    $senha = $_POST['senha'] ?? '';

    // Busca o usuário pelo e-mail
    $stmt = $conn->prepare('SELECT id, nome, senha FROM usuarios WHERE email = ?');
    $stmt->bind_param('s', $email);
    $stmt->execute();
    $usuario = $stmt->get_result()->fetch_assoc();
    $stmt->close();

    // Confere a senha digitada com o hash guardado
    if ($usuario && password_verify($senha, $usuario['senha'])) {
        session_regenerate_id(true);
        $_SESSION['usuario_id']   = $usuario['id'];
        $_SESSION['usuario_nome'] = $usuario['nome'];

        header('Location: index.php');
        exit;
```

```
    }

    // A mesma mensagem para e-mail inexistente e senha errada
    $erro = 'E-mail ou senha incorretos.';
}

$tituloPagina = 'Entrar';
require 'topo.php';
?>
<section class="card card-auth">
    <h1>Entrar</h1>

    <?php if (($_GET['cadastro'] ?? '') === 'ok'): ?>
        <div class="alerta alerta-sucesso">Conta criada! Entre com seu
e-mail e senha.</div>
    <?php endif; ?>

    <?php if ($erro !== ''): ?>
        <div class="alerta alerta-erro"><?= e($erro) ?></div>
    <?php endif; ?>

    <form method="post" action="login.php">
        <div class="campo">
            <label for="email">E-mail</label>
            <input type="email" id="email" name="email" value="<?=
e($email) ?>" required>
        </div>
        <div class="campo">
            <label for="senha">Senha</label>
            <input type="password" id="senha" name="senha" required>
        </div>
        <button type="submit" class="btn btn-primario
btn-bloco">Entrar</button>
    </form>

    <p class="texto-form">Ainda não tem conta? <a
href="cadastro.php">Cadastre-se</a></p>
```

```
</section>

<?php require 'rodape.php'; ?>
```

Como funciona:

1. O PHP busca o usuário pelo e-mail com um prepared statement. `fetch_assoc()` devolve `null` se não existir.
2. `password_verify()` compara a senha digitada com o hash do banco, sem nunca "desfazer" o hash.
3. Em caso de sucesso, guardamos o `id` e o `nome` em `$_SESSION`. É a presença de `usuario_id` na sessão que significa "está logado" (veja `esta_logado()` em `funcoes.php`).
4. A mensagem de erro é a mesma para e-mail inexistente e senha errada, para não revelar quais e-mails têm contato.
5. O parâmetro `?cadastro=ok` na URL só serve para exibir o aviso verde após o cadastro.

logout.php

```
<?php
require_once 'funcoes.php';


// Limpa os dados da sessão
$_SESSION = [];


// Apaga também o cookie da sessão no navegador
if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000, $p['path'],
$p['domain'], $p['secure'], $p['httponly']);
}


// Destrói a sessão no servidor
session_destroy();


header('Location: login.php');
exit;
```

O logout esvazia `$_SESSION`, remove o cookie do navegador e destrói a sessão no servidor. O usuário volta ao login e qualquer tentativa de abrir `index.php` será barrada por `exigir_login()`.

O CRUD de tarefas

Cada operação do CRUD é uma página que responde a um comando SQL. Todas começam com `exigir_login()` e filtram por `usuario_id`, de modo que cada usuário só enxerga e altera as próprias tarefas.

Página	Operação	SQL	Método
index.php	Read	SELECT	GET
criar.php	Create	INSERT	GET (formulário) e POST (gravar)
editar.php	Update	UPDATE	GET (formulário preenchido) e POST (gravar)
excluir.php	Delete	DELETE	Apenas POST

index.php (Read)

Lista as tarefas do usuário logado, da mais recente para a mais antiga, e mostra os avisos de sucesso vindos das outras páginas.

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

$usuarioId = $_SESSION['usuario_id'];

$stmt = $conn->prepare(
    'SELECT id, titulo, descricao, status, criado_em
    FROM tarefas
    WHERE usuario_id = ?
    ORDER BY criado_em DESC'
);
$stmt->bind_param('i', $usuarioId);
$stmt->execute();
$tarefas = $stmt->get_result();

// Mensagens de sucesso, escolhidas por uma chave na URL (?msg=criada)
$mensagens = [
    'criada'   => 'Tarefa criada com sucesso.',
    'editada'  => 'Tarefa atualizada com sucesso.',
    'excluida' => 'Tarefa excluída com sucesso.',
];

$msg = $mensagens[$_GET['msg'] ?? ''] ?? '';

$tituloPagina = 'Minhas tarefas';
require 'topo.php';
?>
<div class="cabecalho-pagina">
    <h1>Minhas tarefas</h1>
    <a class="btn btn-primario" href="criar.php">+ Nova tarefa</a>
</div>
```

```php
<?php if ($msg !== ''): ?>
    <div class="alerta alerta-sucesso"><?= e($msg) ?></div>
<?php endif; ?>

<?php if ($tarefas->num_rows === 0): ?>
    <div class="card vazio">Você ainda não tem tarefas. Clique em Nova
tarefa para começar.</div>
<?php else: ?>
    <div class="card">
        <table class="tabela">
            <thead>
                <tr>
                    <th>Tarefa</th>
                    <th>Status</th>
                    <th>Criada em</th>
                    <th>Ações</th>
                </tr>
            </thead>
            <tbody>
            <?php while ($t = $tarefas->fetch_assoc()): ?>
                <tr>
                    <td>
                        <strong><?= e($t['titulo']) ?></strong>
                        <?php if (!empty($t['descricao'])): ?>
                            <div class="descricao"><?=
e($t['descricao']) ?></div>
                        <?php endif; ?>
                    </td>
                    <td>
                        <span class="badge badge-<?= e($t['status'])
?>"><?= e(STATUS_TAREFA[$t['status']]) ?></span>
                    </td>
                    <td><?= date('d/m/Y H:i',
strtotime($t['criado_em'])) ?></td>
                    <td class="acoes">
                        <a class="btn btn-secundario btn-pequeno"
href="editar.php?id=<?= (int) $t['id'] ?>">Editar</a>
                        <form method="post" action="excluir.php"
onsubmit="return confirm('Excluir esta tarefa?');">
                            <input type="hidden" name="id" value="<?=
(int) $t['id'] ?>">
                            <button type="submit" class="btn btn-perigo
btn-pequeno">Excluir</button>
                        </form>
                    </td>
                </tr>
            <?php endwhile; ?>
            </tbody>
        </table>
    </div>
<?php endif; ?>

<?php require 'rodape.php'; ?>
```

Pontos de atenção:

- O WHERE usuario_id = ? É o que impede um usuário de ver as tarefas de outro.

- while ($t = $tarefas->fetch_assoc()) repete o bloco de HTML uma vez por linha do resultado.
- O botão Excluir fica dentro de um <form> com método POST, e o confirm() em JavaScript solicita confirmação antes de enviar.
- (int) no id garante que só um número vá para a URL e para o formulário.

form_tarefa.php (formulário compartilhado)

Criar e editar usam os mesmos campos. Em vez de repetir o HTML, ele fica em um arquivo incluído por ambas as páginas. Espera três variáveis prontas: $tarefa, $erros e $textoBotao.

```
<?php
// form_tarefa.php - campos do formulário usados por criar.php e
editar.php
// Variáveis esperadas: $tarefa (array), $erros (array) e $textoBotao
(string)
?>
<?php if (!empty($erros)): ?>
    <div class="alerta alerta-erro">
        <ul>
            <?php foreach ($erros as $erro): ?>
                <li><?= e($erro) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

<form method="post">
    <div class="campo">
        <label for="titulo">Título</label>
        <input type="text" id="titulo" name="titulo" maxlength="150"
value="<?= e($tarefa['titulo']) ?>" required>
    </div>

    <div class="campo">
        <label for="descricao">Descrição</label>
        <textarea id="descricao" name="descricao" rows="4"><?=
e($tarefa['descricao']) ?></textarea>
    </div>

    <div class="campo">
        <label for="status">Status</label>
        <select id="status" name="status">
            <?php foreach (STATUS_TAREFA as $valor => $rotulo): ?>
                <option value="<?= e($valor) ?>" <?= $tarefa['status']
=== $valor ? 'selected' : '' ?>>
                    <?= e($rotulo) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>

    <div class="form-acoes">
        <button type="submit" class="btn btn-primario"><?=
e($textoBotao) ?></button>
        <a class="btn btn-secundario" href="index.php">Cancelar</a>
    </div>
```

```
</form>
```

O `<form method="post">` sem `action` envia os dados para a própria URL atual. Isso o mantém `?id=` na edição.

criar.php (Create)

```
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

$erros  = [];
$tarefa = ['titulo' => '', 'descricao' => '', 'status' => 'pendente'];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $tarefa['titulo']    = trim($_POST['titulo'] ?? '');
    $tarefa['descricao'] = trim($_POST['descricao'] ?? '');
    $tarefa['status']    = $_POST['status'] ?? 'pendente';
    $erros = validar_tarefa($tarefa);

    if (empty($erros)) {
        $usuarioId = $_SESSION['usuario_id'];

        $stmt = $conn->prepare(
            'INSERT INTO tarefas (usuario_id, titulo, descricao,
status) VALUES (?, ?, ?, ?)'
        );
        $stmt->bind_param('isss', $usuarioId, $tarefa['titulo'],
$tarefa['descricao'], $tarefa['status']);
        $stmt->execute();
        $stmt->close();

        header('Location: index.php?msg=criada');
        exit;
    }
}

$tituloPagina = 'Nova tarefa';
$textoBotao   = 'Salvar tarefa';
require 'topo.php';
?>
<section class="card">
    <h1>Nova tarefa</h1>
```

```
    <?php require 'form_tarefa.php'; ?>
</section>
<?php require 'rodape.php'; ?>
```

O usuário dono da tarefa vem sempre da sessão, nunca de um campo do formulário. Assim ninguém consegue criar tarefas em nome de outra pessoa.

editar.php (Update)

```
<?php

// Inicia a sessão uma única vez (necessário para o $_SESSION
funcionar)
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// Status permitidos para uma tarefa: valor no banco => texto exibido
const STATUS_TAREFA = [
    'pendente'  => 'Pendente',
    'andamento' => 'Em andamento',
    'concluida' => 'Concluída',
];

// Retorna true se existe um usuário logado
function esta_logado(): bool
{
    return isset($_SESSION['usuario_id']);
}

// Envia para o login quem não estiver logado
function exigir_login(): void
{
    if (!esta_logado()) {
        header('Location: login.php');
        exit;
    }
}

// Escapa um texto antes de exibi-lo no HTML (protege contra XSS)
function e(?string $texto): string
{
    return htmlspecialchars($texto ?? '', ENT_QUOTES, 'UTF-8');
}

// Valida os dados de uma tarefa e devolve a lista de erros (vazia se
estiver tudo certo)
function validar_tarefa(array $dados): array
{
    $erros = [];

    if (empty($dados['titulo'])) {
        $erros[] = 'Informe o título da tarefa.';
    } elseif (mb_strlen($dados['titulo']) > 150) {
        $erros[] = 'O título pode ter no máximo 150 caracteres.';
    }
```

```
    if (!array_key_exists($dados['status'], STATUS_TAREFA)) {
        $erros[] = 'Status inválido.';
    }

    return $erros;
}
```

O fluxo tem duas fases. No GET, a tarefa é buscada e o formulário abre já preenchido. No POST, os campos recebem o que foi digitado, são validados e então o `UPDATE` grava. O `AND usuario_id = ?` Aparece nas duas consultas para proteger as tarefas alheias, mesmo que alguém troque o número na URL.

excluir.php (Delete)

Não tem tela: recebe o `id` por POST, apaga e volta para a lista.

```
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

// Só aceita POST: um link (GET) nunca deve apagar dados
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: index.php');
    exit;
}

$id    =(int)($_POST['id'] ?? 0);
$usuarioId = $_SESSION['usuario_id'];

$stmt = $conn->prepare('DELETE FROM tarefas WHERE id = ? AND usuario_id
= ?');
$stmt->bind_param('ii', $id, $usuarioId);
$stmt->execute();
$stmt->close();

header('Location: index.php?msg=excluida');
exit;
```

Excluir por POST evita que um robô de busca ou uma pré-visualização de link apague tarefas ao "visitar" um endereço.

Testando tudo

Com os 13 arquivos no lugar, abra http://localhost/crud_tarefas/cadastro.php e percorra esta lista; se todos os itens funcionarem, o CRUD está completo.

☐ Criar uma conta em cadastro.php (teste também senhas diferentes e e-mail repetido, que devem mostrar erros)
☐ No phpMyAdmin, abrir a tabela usuários e conferir que senha guarda um texto longo começando com $2y$, e não a senha digitada
☐ Entrar com a senha errada (mensagem de erro) e depois com a certa (cai na lista)
☐ Criar uma tarefa e vê-la aparecer na lista
☐ Editar a tarefa, mudando o status, e conferir a etiqueta colorida
☐ Excluir a tarefa e confirmar a pergunta do navegador
☐ Clicar em Sair e tentar abrir index.php: deve voltar ao login
☐ Criar uma segunda conta e confirmar que ela não vê as tarefas da primeira

Erros comuns

Mensagem ou sintoma	Causa provável	Solução
Access denied for user 'root'@'localhost'	O MySQL tem uma senha diferente da padrão	Ajuste $pass em conexao.php
Unknown database 'crud_tarefas'	O script SQL não foi executado, ou o nome está diferente	Rode o script da seção do banco e confira $banco
Table 'crud_tarefas.tarefas' doesn't exist	As tabelas não foram criadas	Execute de novo o script SQL
Cannot modify header information - headers already sent	Algo foi exibido antes de um header()	Mantenha a lógica no começo do arquivo, sem linhas ou espaços antes de <?php
Acentos aparecem como Ã§ ou Ã£	Arquivo fora de UTF-8	No VS Code, salve em UTF-8; confira set_charset e <meta charset>
Página em branco	Erro de PHP não exibido	Veja o log de erros do Apache (botão Logs do XAMPP)
Fica voltando para o login	A sessão não foi criada	Confira se o login.php define $_SESSION['usuario_id'] e se funcoes.php é incluído